Relatório — Parte 2: transmissão para a nuvem e visualização (Fog/Cloud Computing)

Projeto: CardioIA Conectada — Fase 3
Aluno: Marlon Paulino Marinho — RM566793
Simulação: https://wokwi.com/projects/479676068207522817
Arquivos: `esp32/sketch.ino` e `node-red/flow_cardioia.json`

1. Visão geral da arquitetura

```
[ESP32 / Wokwi] --publica--> [HiveMQ] --assina--> [Node-RED] --> [Dashboard]
 DHT22 + botão              broker.hivemq.com      regras de     gráfico, gauge,
 buffer offline (Edge)      porta 1883             alerta (Fog)  texto e LED
```

Cada camada tem um papel:

Camada	Onde roda	O que faz
Edge	ESP32 (Wokwi)	Lê sensores, calcula BPM, acende o alerta local e guarda leituras quando está offline
Mensageria	HiveMQ (broker MQTT)	Recebe as mensagens publicadas e entrega a quem assinou o tópico
Fog	Node-RED no computador local	Assina os tópicos, separa os sinais, reaplica as regras de alerta e alimenta o painel
Visualização	Node-RED Dashboard	Gráfico em tempo real, gauge, texto de estado e LED virtual

2. Fluxo de comunicação MQTT

Por que MQTT. É um protocolo leve de publicação e assinatura, feito para aparelhos com pouca bateria e rede instável. O ESP32 não precisa saber quem vai ler os dados: ele só publica num **tópico**. Quem quiser recebe por assinatura. Cabeçalho mínimo, conexão sempre aberta e reconexão simples.

Broker. Usamos o broker público da HiveMQ (`broker.hivemq.com`, porta 1883, sem usuário e senha). Ele funciona dentro do simulador pela rede `Wokwi-GUEST` e dispensa cadastro. A troca para o **HiveMQ Cloud** (cluster privado, TLS na porta 8883, usuário com permissão em *Access Management*) muda só três linhas: host, porta e `WiFiClientSecure` com usuário e senha. Veja a seção 5.

Tópicos. Um prefixo com o RM evita misturar dados com outros alunos no broker público.

Tópico	Quem publica	Conteúdo	Retain
`cardioia/rm566793/vitais`	ESP32	Uma leitura por mensagem (JSON)	Não
`cardioia/rm566793/status`	ESP32	Estado da conexão e ocupação do buffer	Sim

Formato da mensagem (`vitais`):

```
{"id":"cardioia-rm566793","seq":42,"temp":36.6,"umid":55,"bpm":84,"alerta":"OK","atraso_s":0}
```

- `seq`: número da leitura, que permite achar buracos e evitar duplicidade.
- `atraso_s`: há quantos segundos a leitura foi medida. É 0 quando o dado é ao vivo e maior quando veio do buffer offline.
- `alerta`: classificação já feita no ESP32 (Edge), para o caso de a nuvem estar fora.

Sequência de uma mensagem:

1. O ESP32 conecta no Wi-Fi do simulador e abre a sessão MQTT com um `clientId` único (`cardioia-rm566793-<MAC>`).
2. Na conexão, registra uma **mensagem de testamento** (*Last Will*) no tópico `status`: `{"estado":"desconectado"}`. Se a placa cair sem avisar, o próprio broker publica esse aviso e o painel mostra o dispositivo desconectado.
3. A cada leitura, o ESP32 publica no tópico `vitais` com QoS 0.
4. O broker entrega a mensagem ao Node-RED, que assinou o mesmo tópico.
5. Em caso de falha no `publish`, a leitura **não sai do buffer** e é reenviada na próxima volta (seção 3 do relatório da Parte 1).
6. A cada 5 s sem conexão, o ESP32 tenta reconectar sem travar a leitura dos sensores.

Por que QoS 0. Com QoS 0 o envio é "no máximo uma vez", o mais leve. A garantia de entrega fica no próprio ESP32: a leitura só é apagada depois do `publish` aceito, e o `seq` permite detectar perdas. Num produto real o próximo passo é QoS 1 ("pelo menos uma vez"), com o `seq` servindo para descartar duplicatas.

3. Configuração do dashboard no Node-RED

Instalação. Node.js LTS e `npm install -g --unsafe-perm node-red`. No editor (`http://127.0.0.1:1880`), em *Manage palette*, instala-se o pacote **node-red-dashboard**. O fluxo é importado pelo menu *Import* com o arquivo `node-red/flow_cardioia.json`. O painel abre em `http://127.0.0.1:1880/ui`.

Nós do fluxo:

Nó	Tipo	Configuração
ESP32 -> vitais	`mqtt in`	Broker HiveMQ, tópico `cardioia/rm566793/vitais`, saída *parsed JSON*
Separar sinais e checar limites	`function` (5 saídas)	Separa BPM e temperatura para o gráfico, temperatura para o gauge, texto de estado e sinal do LED
Gráfico BPM e temperatura	`ui_chart` (linha)	Duas séries (BPM em vermelho, temperatura em azul), janela de 10 minutos, eixo 0–200
Gauge temperatura	`ui_gauge`	34 a 42 °C. Faixas: azul até 37,5, verde até 38 e vermelho acima de 38 (febre)
Texto de alerta	`ui_text`	"● Normal" ou "⚠ FEBRE / TAQUICARDIA / BRADICARDIA", com nº da leitura e indicação "veio do buffer"
LED de alerta	`function` + `ui_template`	Círculo verde (normal) ou vermelho piscando (alerta)
ESP32 -> status	`mqtt in` + `ui_text`	Mostra online/desconectado, ocupação do buffer e leituras perdidas
Ver JSON recebido	`debug`	Mostra cada mensagem bruta na barra lateral, para conferência

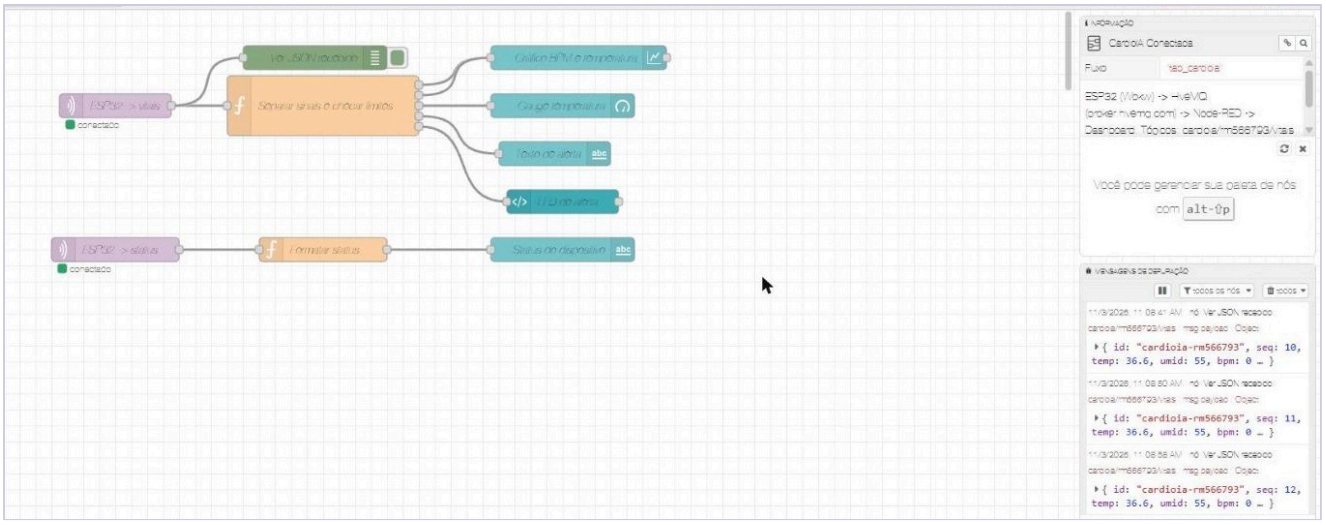
Regras de alerta (definidas pelo projeto):

Parâmetro	Limite	Alerta
Temperatura	> 38,0 °C	FEBRE
Batimentos	> 120 bpm	TAQUICARDIA
Batimentos	< 50 bpm (com pulso detectado)	BRADICARDIA

A regra existe em dois lugares de propósito. No **ESP32** ela acende o LED físico mesmo sem internet (Edge). No **Node-RED** ela é recalculada sobre o dado recebido (Fog). Assim os limites podem mudar no painel sem regravar a placa, e o alerta continua funcionando se uma das camadas falhar.

4. Evidências

Teste feito em 03/11/2026, com o ESP32 rodando no Wokwi e o Node-RED 5.0.9 rodando localmente (node-red-dashboard 3.6.6). Os dois nós MQTT ficaram com status **conectado**, e a barra de *debug* mostrou cada JSON recebido. Todas as imagens estão na pasta `prints/` do repositório.



Fluxo no Node-RED: nós MQTT conectados e JSON chegando na barra de debug



Dashboard em alerta de FEBRE: gauge no vermelho (38,9 °C) e LED virtual aceso



Dashboard em alerta de TAQUICARDIA: BPM acima de 120 no gráfico



Dashboard de volta ao normal: temperatura 36,8 °C, LED verde, buffer 0/3000 e nenhuma leitura perdida

Observação sobre o gráfico. O primeiro pico de BPM aparece quase vertical porque as 20 leituras guardadas offline chegaram de uma vez na sincronização. O gráfico usa a hora de chegada. O texto de estado indica quando a leitura "veio do buffer", com base no campo `atraso_s`.

5. Eficiência, segurança e boas práticas em IoT médico

- **Eficiência.** Mensagens de cerca de 100 bytes, sem cabeçalho HTTP, e uma conexão MQTT reaproveitada. No produto real, 1 leitura a cada 10 s dá cerca de 860 KB por dia por paciente.

- **Privacidade (LGPD).** Dado de saúde é dado pessoal sensível. A mensagem não leva nome nem CPF, só um identificador do aparelho (`cardioia-rm566793`). A ligação aparelho–paciente fica no sistema do hospital, fora do broker.
- **Segurança na versão de produção.** O broker público serve só para demonstração: qualquer pessoa que souber o tópico consegue ler. Em produção:
- usar HiveMQ Cloud com **TLS (porta 8883)**, usuário e senha por dispositivo, e permissão restrita ao próprio tópico;
- guardar credenciais fora do código, nunca no repositório público.
- **Confiabilidade.** Alerta local no Edge, *Last Will* para detectar queda e numeração `seq` para auditoria de perdas.
- **Limitação do protótipo.** O BPM é simulado por toques num botão, não por um sensor de pulso real (como o MAX30102). Os limites de alerta são didáticos e não substituem critério médico.